



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

Lenguajes de Programación

Proyecto 2: Tipos dependientes y demostraciones computacionales.

PROFESOR

Manuel Soto Romero

AYUDANTES

Diego Méndez Medina
José Alejandro Pérez Márquez
Erick Daniel Arroyo Martínez
Mauro Emiliano Chávez Zamora

ALUMNOS

López Rodríguez Leslie Renée - 321171915
Marentes Mosqueda Bruno Sebastián - 321260606

14 de diciembre de 2025

Índice general

1. Introducción	3
2. Tipos dependientes: Fundamentos teóricos	4
2.1. Definición	4
2.2. Tipos dependientes vs tipos simples	4
2.3. Propiedades como tipos	5
2.4. Funciones totales y demostraciones como programas	5
2.5. Verificación automática por el compilador	6
3. Expresión de propiedades matemáticas en Agda	7
3.1. Inducción en Agda	7
4. Ejemplo práctico: reverse preserva la longitud	8
4.1. Definición de listas en Agda	8
4.2. Definición de length	8
4.3. Definición de reverse	9
4.4. Ejercicio: Implementar un ejemplo en Agda donde el tipo garantice que una función reverse conserva la longitud de la lista.	9
5. Conclusiones	14

Capítulo 1

Introducción

En el mundo de la programación solemos pensar en los tipos como etiquetas que nos dicen que clase de datos maneja un programa: números, cadenas, listas, etc., sin embargo, los tipos dependientes como en *Agda*, *Idris* y *Coq*, que representan un avance pues permiten que los tipos dependan directamente de los valores, así propiedades lógicas y matemáticas pueden expresarse dentro de la propia definición de una función, logrando que ciertos aspectos del comportamiento de un programa se garanticen por el sistema de tipos mismo. Por consiguiente, tenemos que el compilador no solo revisa la sintaxis, sino que actúa como un auténtico verificador de pruebas.

El objetivo de este proyecto es explorar cómo los tipos dependientes pueden utilizarse para expresar y verificar propiedades matemáticas dentro del propio programa, integrando los conceptos teóricos con una demostración práctica realizada en Agda. Para esto, estudiaremos los fundamentos de los tipos dependientes, la noción de funciones totales y la interpretación de las demostraciones como programas. Posteriormente, se presenta un ejemplo concreto en el que mostraremos la función *reverse*, aplicada a una lista, preservando su longitud, una propiedad que puede ser formalizada y verificada utilizando el sistema de tipos de Agda.

Capítulo 2

Tipos dependientes: Fundamentos teóricos

2.1. Definición

Entendemos por tipos dependientes

Los tipos dependientes son tipos cuya forma varía en función de valores. A diferencia de los tipos simples, como productos, funciones o números naturales, donde la estructura es fija, en los tipos dependientes la especificación del tipo puede depender de un término.

Así se permite expresar propiedades más precisas sobre los datos directamente en el sistema de tipos, de tal manera que la validez de ciertos programas se puede garantizar por construcción. Hofmann(1997)[?], señala que los tipos dependientes son *"types which, unlike simple types, such as products, function spaces, or natural numbers, depend on or vary with values"*.

Así bajo este enfoque, los tipos los podemos definir tomando como parámetros valores concretos. Algunos ejemplos son los vectores donde el tamaño queda determinado en el tipo, las funciones cuyo tipo incorpora condiciones que deben cumplirse antes de ejecutarlas, o estructuras cuyos requisitos dependen de información disponible durante la etapa de tipificación.

2.2. Tipos dependientes vs tipos simples

Normalmente en los lenguajes de programación, utilizamos los tipos simples para describir el valor general de las variables, como son los booleanos (Bool), enteros (Int), cadenas (String), etcétera. Sin embargo, con estos tipos estamos limitados a no poder expresar libremente diferentes propiedades en la propia estructura del tipo, es decir, no podemos plantear explícitamente si un arreglo de enteros tiene una longitud de n elementos, mantiene una propiedad en su entrada o si ciertos valores deben cumplir alguna restricción que declaremos.

Entonces, esta falta de esta expresividad que representan los tipos simples, podemos llegar a tener problemas de no poder garantizar los comportamientos que deseamos de manera más precisa en tiempo de compilación de nuestro lenguaje. Por otro lado, los tipos dependientes, nos ayudan a ampliar este sistema de tipos simples que manejábamos permitiéndonos añadirle más información directamente a la definición del tipo, es decir, a su propia estructura, lo que nos ayuda a describir de mejor manera ciertas características, restricciones o invariantes intrínsecas de nuestros datos. De esta manera, podemos decir,

que mientras que los tipos simples únicamente nos indican lo que representa en general un valor, los tipos dependientes pueden describir no sólo el valor, sino que también una propiedad sobre este, así en el mismo tipo estamos especificando y verificando lo que queremos declarar.

2.3. Propiedades como tipos

Las propiedades como tipos, es la idea central de una correspondencia entre la lógica y teoría de tipos: una proposición o propiedad que queremos demostrar se representa como un tipo, y una prueba de esa proposición corresponde a un valor que habita ese tipo.

Así la interpretación *propositions-as-types*, afirma:

- Una proposición lógica se interpreta como un tipo.
- Una prueba de esa proposición se interpreta como un programa (o un valor) que es miembro de ese tipo.
- La validez de la proposición queda garantizada cuando el tipo tiene al menos un valor.

Así implica que las propiedades matemáticas o lógicas, cómo “*x es primo*”, “*la longitud de una lista es n*” y “*un número es distinto a cero*”, se pueden modificar de forma directa en el sistema de tipos dependientes.

2.4. Funciones totales y demostraciones como programas

Una de las principales características de los lenguajes que usan tipos dependientes es que solamente permiten la definición de funciones totales. Entendemos por función total a aquellas funciones que nos devuelven un resultado para todas las entradas de su dominio y además el cálculo de esta función siempre termina en un número finito de pasos, estas condiciones son indispensables para ver la interpretación de Curry-Howard, la cual nos dice, que las funciones corresponden a demostraciones lógicas y, por lo tanto, deben ser constructivas y garantizar terminación.[3]

Norell nos dice que para que se acepte una definición en lenguajes como Agda, depende precisamente de que el sistema pueda verificar que la función termina. Entonces, el compilador va ir analizando cada llamada recursiva que se realiza, para verificar que efectivamente el argumento que va tomando a su vez va disminuyendo, asegurándose así que la evaluación no nos produce bucles infinitos. Por lo tanto, al realizarse esta verificación el sistema mantiene su consistencia, en el caso que una función no pudiese terminar correctamente, esto nos daría una demostración inválida o como mínimo incompleta.[2]

Ya que tenemos claro que la totalidad es una propiedad realmente importante, podemos ver que es precisamente este requisito el que nos permite interpretar las funciones como demostraciones dentro del sistema de tipos. Bajo el principio *propositions-as-types*, cada proposición se representa mediante un tipo y cada prueba corresponde a un valor que tiene este tipo; así, podemos escribir una función que equivale a construir una demostración. Hofmann nos dice que esta identificación entre programas y demostraciones solo es válida si el sistema nos garantiza que todos los términos son normales, es decir, que cualquier paso en la evaluación termina produciendo un valor concreto. Por lo tanto, la verificación

de tipos actúa como un mecanismo que nos confirma estructuralmente la corrección lógica de las construcciones definidas en el lenguaje.

2.5. Verificación automática por el compilador

En los sistemas con tipos dependientes, la verificación de tipos realizada por el compilador no solo revisa que los términos estén bien formados, sino que también verifica que las definiciones cumplan con las reglas de productividad, normalización y totalidad. Norell(2007)[2] menciona que la aceptación de una definición depende de que el sistema pueda garantizar que cualquier función definida en el lenguaje termina en un número finito de pasos.

Como bajo la interpretación de *propositions-as-types*, cada proposición se representa como un tipo y cada prueba como un término que lo habita. Esta comprobación de tipos garantiza de manera estructural la validez de las propiedades que se expresan en el programa. Hofmann(1997)[1] explica que la corrección de las pruebas queda asegurada por las reglas de formación e indiferencia del sistema, de modo que cualquier término aceptado por el verificador de tipos constituye efectivamente una demostración válida. Así, de manera complementaria, Sørensen y Urzyczyn[3] destacan que la verificación de tipos se corresponde directamente con la verificación de derivaciones lógicas, lo que permite interpretar el proceso de compilación como una forma de comprobación automática. Por lo tanto tenemos que la comprobación del programa no solo valida aspectos sintácticos, sino que también garantiza que las funciones cumplan con las propiedades especificadas en sus tipos, en consecuencia el compilador actúa como un mecanismo formal que impide ejecutar programas que no satisfacen las condiciones declaradas.

Capítulo 3

Expresión de propiedades matemáticas en Agda

Agda es un lenguaje de programación funcional y un asistente de pruebas basado en teoría de tipos dependientes. Permite escribir programas y, al mismo tiempo, demostrar propiedades sobre ellos dentro del propio sistema de tipos. Una consecuencia central del tipo de identidad es el principio de sustitución, como REFL verifica que dos términos son indistinguibles, Agda permite reemplazar un término por otro en cualquier parte donde aparezca. Esta regla explicada por Sørensen y Urzyczyn[3], refleja la idea lógica de la igualdad como congruencia.

Norell(2007)[2] menciona que en las funciones recursivas sobre listas y las pruebas que establecen propiedades acerca de ellas siguen patrones, estructuralmente idénticos.

3.1. Inducción en Agda

Primero, debemos entender que las pruebas en Agda se construyen por inducción estructural, utilizando *pattern matching*, esto es porque todos los tipos de datos inductivos se construyen mediante un conjunto finito de constructores. Así cualquier término que pertenezca al tipo debe haberse formado a partir de dichos constructores, permitiendo que las pruebas se organicen siguiendo la misma estructura. Según Norell(2007)[2], esta correspondencia nos garantiza que cada demostración sea completa, ya que abarca todas las formas posibles que un valor puede tener.

```
sym : (x y : A) -> x ≡ y -> y ≡ x
sym x .x refl = refl
```

Este estilo de prueba refleja la idea de que demostrar propiedades en Agda equivale a escribir programas que manipulan constructores inductivos. Norell [2]

Capítulo 4

Ejemplo práctico: reverse preserva la longitud

En este capítulo desarrollamos una demostración formal en Agda de que la función `reverse` preserva la longitud de una lista. Este ejemplo sirve como caso de estudio para mostrar cómo Agda permite especificar tipos dependientes, definir funciones recursivas y certificar propiedades mediante pruebas constructivas verificadas por el compilador.

4.1. Definición de listas en Agda

Agda nos permite especificar tipos inductivos mediante `data`. El tipo de listas está parametrizado por el tipo de sus elementos y se define inductivamente a partir de dos constructores: la lista vacía y el constructor que agrega un elemento al frente de una lista ya existente.

```
data List (A : Set) : Set where
  []      : List A
  _::__   : A -> List A -> List A
```

El constructor `[]` representa la lista vacía, mientras que el constructor `_::__` toma un elemento de tipo `A` y una lista de `A` para producir una nueva lista. De esta forma se permite definir funciones por recursión estructural sobre listas.

4.2. Definición de length

La función `length` en Agda calcula la longitud de una lista contando el número de elementos que tiene. Se define siguiendo la estructura del tipo `List`, lo que significa que se programa por recursión estructural, por lo tanto, cada caso de la función corresponde directamente a uno de los constructores del tipo listas.

```
length : {A : Set} -> List A -> Nat
length [] = zero
length (_ :: xs) = suc (length xs)
```

En el caso base, la lista vacía `[]` tiene longitud `zero`, lo que coincide con su definición. En el caso inductivo, una lista de la forma `x :: xs` contiene al menos un elemento; por ello, la longitud se obtiene

aplicando `suc` (constructor que representa el sucesor de un número natural) a la longitud de la cola `xs`. De esta manera, la llamada recursiva opera sobre una subestructura estrictamente menor, lo que nos garantiza que la función termina, tal como habíamos visto en la sección: *Funciones totales y demostraciones como programas* para los tipos dependientes.

4.3. Definición de reverse

En nuestro desarrollo, trabajamos con una definición propia del tipo `List`, construida inductivamente mediante dos constructores: la lista vacía `[]` y el operador `_::__` que inserta un elemento al inicio de la lista. Sobre esta estructura, definimos la función `reverse`, cuya finalidad es invertir el orden de los elementos de la lista dada.

```
reverse : {A : Set} -> List A -> List A
reverse [] = []
reverse (x :: xs) = reverse xs ++ (x :: [])
```

Como podemos observar, vemos que la definición es recursiva:

- En el **caso base**, si la lista está vacía, la inversión también es una lista vacía.
- En el **caso inductivo**, para una lista `x :: xs`, se invierte primero la cola `xs` y luego se concatena el elemento `x` al final mediante la operación `++`.

Esta definición asegura que cada llamada recursiva trabaja sobre una lista estrictamente más pequeña, lo cual garantiza la terminación del proceso. Además, al estar expresada dentro del sistema de tipos de Agda, queda certificado que `reverse` es una función total y correctamente estructurada.

4.4. Ejercicio: Implementar un ejemplo en Agda donde el tipo garantice que una función reverse conserva la longitud de la lista.

El objetivo de este ejemplo es implementar la función `reverse` sobre listas en Agda de manera que su tipo garantice que la longitud de la lista se conserva. Esto significa que no solo definimos la función, sino que el tipo dependiente asegura automáticamente que `length (reverse xs) = length xs` para cualquier lista `xs`.

Importación de módulos y fundamentos lógicos

```
module ReverseLength where

open import Agda.Builtin.Equality
open import Agda.Builtin.Nat
open import Relation.Binary.PropositionalEquality using (cong; trans; sym)
```

Para que Agda pueda verificar nuestra demostración, necesitamos importar ciertas definiciones básicas que actúan como los "bloques de construcción" de nuestra prueba. A continuación explicamos qué función cumple cada una:

■ `open import Agda.Builtin.Equality`

Este módulo es indispensable para hablar de igualdad. En Agda, decir que dos cosas son iguales no es una simple comparación (como en otros lenguajes que devuelven *true* o *false*), sino que es un tipo de dato que debemos construir. Al importar esto, obtenemos acceso al símbolo $\equiv$ para escribir nuestro teorema y al constructor `refl`, que utilizamos para probar los casos base donde dos términos son idénticos.

■ `open import Agda.Builtin.Nat`

Aquí importamos los números naturales. Como nuestra función `length`, que cuenta elementos, necesita devolver un valor de tipo `Nat`. Este módulo le enseña a Agda qué son el `zero` y la función sucesor (`suc`), permitiéndonos realizar operaciones aritméticas básicas durante la recursión.

■ `open import Relation.Binary.PropositionalEquality using (cong; trans; sym)`

De este módulo tomamos tres herramientas lógicas que nos ayudan a dar pasos en la demostración:

- **cong (Congruencia):** Es la regla que dice: "si dos valores son iguales, y les aplicamos la misma función, el resultado sigue siendo igual". La usamos en el paso inductivo para agregar `suc` a ambos lados de la igualdad.

$$(x \equiv y \rightarrow f(x) \equiv f(y))$$

- **trans (Transitividad):** Nos permite encadenar igualdades. Si probamos que A es igual a B , y que B es igual a C , `trans` nos deja concluir que A es igual a C . Es fundamental para unir nuestros lemas auxiliares.

$$(a \equiv b \wedge b \equiv c \rightarrow a \equiv c)$$

- **sym (Simetría):** Nos permite "voltear" una igualdad (decir que si $x \equiv y$, entonces $y \equiv x$). Aunque a veces no se usa explícitamente, es útil si un lema nos da la igualdad en el orden inverso al que necesitamos.

$$(x \equiv y \rightarrow y \equiv x)$$

Definimos el tipo inductivo en lista, tenemos una lista vacía, o un elemento seguido de otra lista.

```
data List (A : Set) : Set where
  [] : List A
  _::_ : A -> List A -> List A
```

Luego definimos la concatenación de listas como función total.

```
_++_ : {A : Set} -> List A -> List A -> List A
[] ++ ys = ys
(x :: xs) ++ ys = x :: (xs ++ ys)
```

Ahora definimos `reverse`.

```
reverse : {A : Set} -> List A -> List A
reverse [] = []
reverse (x :: xs) = reverse xs ++ (x :: [])
```

Y definimos length.

```
length : {A : Set} -> List A -> Nat
length [] = zero
length (_ :: xs) = suc (length xs)
```

Lemas necesarios para la demostración. Este el nos ayuda a demostrar que la longitud de la concatenación es la suma de los longitudes.

```
length-++ : {A : Set} -> (xs ys : List A) ->
            length (xs ++ ys) ≡ length xs + length ys
length-++ [] ys = refl
length-++ (x :: xs) ys =
  cong suc (length-++ xs ys)
```

Luego este lema dice que suma 1 es igual a que apliques suc.

```
+suc : (n : Nat) -> n + 1 ≡ suc n
+suc n = refl
+suc (suc n) = cong suc (+suc n)
```

Demostración principal

```
reverse-length :
  {A : Set} -> (xs : List A) ->
  length (reverse xs) ≡ length xs
reverse-length [] = refl
reverse-length (x :: xs) =
  trans
    (length-++ (reverse xs) (x :: []))
    (trans
      (cong (λ n -> n + 1) (reverse-length xs))
      (+suc (length xs)))
```

1. Tipo de la función

La función no produce un número, sino una *prueba* cuyo tipo afirma la propiedad matemática:

$$\text{length} (\text{reverse } xs) \equiv \text{length } xs.$$

Este tipo expresa la propiedad directamente dentro del programa utilizando tipos dependientes.

2. Caso base

```
reverse-length [] = refl
```

Cuando $xs = []$, ambas expresiones se reducen a 0, por lo que la igualdad se prueba con `refl`. Este paso cierra la base inductiva.

3. Caso inductivo

Queremos demostrar:

$$\text{length} (\text{reverse} (x :: xs)) \equiv \text{length} (x :: xs).$$

Vamos a aplicar varios pasos conectados mediante `trans`, que expresa transitividad de la igualdad.

3.1. Primer paso: aplicar el lema length-++

```
length-++ (reverse xs) (x :: [])
```

Este lema transforma:

$$\text{length} (\text{reverse} xs ++ (x :: []))$$

en:

$$\text{length} (\text{reverse} xs) + \text{length} (x :: []).$$

Dado que $\text{length} (x :: []) = 1$, el término se convierte en:

$$\text{length} (\text{reverse} xs) + 1.$$

3.2. Segundo paso: usar la hipótesis inductiva

```
cong (\n -> n + 1) (reverse-length xs)
```

La hipótesis inductiva nos dice:

$$\text{length} (\text{reverse} xs) \equiv \text{length} xs.$$

Pero en este punto de la prueba necesitamos la misma igualdad, pero con “+1” aplicado a ambos lados. Esto se logra con `cong`, que formaliza:

$$a \equiv b \Rightarrow f(a) \equiv f(b).$$

Aquí $f(n) = n + 1$, por lo que obtenemos:

$$\text{length} (\text{reverse} xs) + 1 \equiv \text{length} xs + 1.$$

3.3. Tercer paso: aplicar el lema +-suc

```
+-suc (length xs)
```

El lema +-suc afirma:

$$n + 1 \equiv \text{suc}(n).$$

Entonces convierte:

$$\text{length } xs + 1$$

en:

$$\text{suc } (\text{length } xs).$$

Y este es exactamente:

$$\text{length } (x :: xs).$$

$\therefore$ se cumple que $\text{length } (\text{reverse } xs) \equiv \text{length } xs$.

Y nos queda de la siguiente manera:

```
reverse-length :
  {A : Set} -> (xs : List A) ->
    length (reverse xs) ≡ length xs
reverse-length [] = refl
reverse-length (x :: xs) =
  trans
    (length-++ (reverse xs) (x :: []))
    (trans
      (cong (λ n -> n + 1) (reverse-length xs))
      (+-suc (length xs)))
```

Al finalizar esta prueba, Agda nos confirma que nuestra propiedad se cumple:

Capítulo 5

Conclusiones

Con la realización de este proyecto, hemos podido comprobar en la práctica cómo los tipos dependientes permiten unificar la programación con la lógica matemática. Más allá de la teoría, trabajar directamente con Agda nos demostró que el compilador actúa como un asistente de demostración, no basta con implementar una función que calcule el resultado correcto, sino que es necesario justificar paso a paso, dentro del mismo código, por qué ese comportamiento es válido.

Un punto crucial del desarrollo fue comprender que propiedades que en papel parecen triviales requieren una justificación explícita formal. Para lograr que la demostración compilara, fue indispensable garantizar la totalidad de las funciones recursivas y construir lemas auxiliares (como `length-++` y `+-suc`) para manejar la inducción estructural. Esto evidencia que, bajo la interpretación de *propositions-as-types*, escribir un programa funcional total equivale realmente a construir una prueba matemática rigurosa.

Finalmente, el proyecto nos muestra la diferencia entre verificar software mediante pruebas unitarias y hacerlo mediante tipos dependientes. Mientras que los tests tradicionales solo cubren casos aislados, nuestra demostración garantiza formalmente que la propiedad:

$$\text{length } (\text{reverse } xs) \equiv \text{length } xs$$

se cumple para cualquier lista posible. En conclusión, este enfoque nos permite cambiar la forma de ver el problema, pasar de buscar errores después de la implementación a comprobar la corrección del programa desde su propia construcción.

Bibliografía

- [1] M. Hofmann, “Syntax and semantics of dependent types,” en *Extensional Constructs in Intensional Type Theory*. Londres: Springer London, 1997, pp. 13–54.
- [2] U. Norell, “Towards a practical programming language based on dependent type theory,” Tesis doctoral, Chalmers University of Technology, Göteborg, Suecia, 2007.
- [3] M. H. B. Sørensen y P. Urzyczyn, *Lectures on the Curry-Howard Isomorphism*, ser. Studies in Logic and the Foundations of Mathematics, vol. 149. Ámsterdam: Elsevier, 2006.